BÁO CÁO THÍ NGHIỆM CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

Cải tiến Hiệu năng MergeSort bằng Kỹ thuật Cutoff

Khảo sát thực nghiệm tìm ngưỡng chuyển đổi tối ưu sang InsertionSort trên kiến trúc JVM

Log Chat với AI: https://gemini.google.com/share/d/1NJO_9LQripO_G2HGHmD9BJuZSOgk7f63?usp=sharing

GitHub Repository: duynqoo/MergeSort_CutOff **Ngôn ngữ cài đặt:** Java & Python

Mục tiêu: Đánh giá hiệu năng & xác định ngưỡng Cutoff tối ưu

---

1. Tóm tắt và Cơ sở Lý thuyết

MergeSort là thuật toán sắp xếp chia để trị kinh điển có độ phức tạp thời gian đảm bảo $O(n \log n)$ trong mọi trường hợp. Tuy nhiên, ở các tầng đệ quy sâu khi kích thước mảng con trở nên nhỏ, chi phí gọi hàm đệ quy liên tục và thao tác sao chép qua mảng phụ `aux[]` làm suy giảm hiệu năng thực tế.

Ngược lại, **InsertionSort** có độ phức tạp $O(n^2)$ nhưng sở hữu hằng số ẩn rất nhỏ, không tốn chi phí gọi hàm đệ quy, sắp xếp tại chỗ (in-place) và tận dụng tốt tính cục bộ không gian của bộ nhớ đệm CPU (L1 Cache). Kỹ thuật **Cutoff** kết hợp ưu điểm của cả hai bằng cách chuyển sang dùng InsertionSort ngay khi kích thước mảng con `(hi − lo + 1) ≤ cutoff`.

> **Kết luận chính từ thực nghiệm:** Ngưỡng `cutoff` tối ưu nằm trong khoảng **15 đến 20** (đặc biệt ổn định quanh mốc **15–16**). Kỹ thuật này giúp giảm thời gian thực thi trung bình **12,31%** ở quy mô 10^6 phần tử, **6,66%** ở quy mô 10^7 phần tử và **3,42%** ở quy mô 10^8 phần tử so với MergeSort thuần túy (`cutoff = 0`).

2. Thiết kế Thí nghiệm và Quy trình Đo đạc

2.1. Môi trường phần cứng thí nghiệm

Toàn bộ quá trình sinh dữ liệu và đo đạc thời gian thực thi được tiến hành trên cùng một thiết bị phần cứng cố định để đảm bảo tính đồng nhất:

- **Thiết bị:** Lenovo Legion R7000 2024
- **Vi xử lý (CPU):** AMD Ryzen 7 8745H
- **Bộ nhớ (RAM):** 16 GB
- **Ổ cứng lưu trữ:** 512 GB SSD
- **Đồ họa (GPU):** NVIDIA GeForce RTX 4060

2.2. Tập dữ liệu thí nghiệm

Bộ dữ liệu được sinh ngẫu nhiên bằng công cụ `DataGenerator.java` với hạt giống cố định (`seed = 42`) để đảm bảo tính tái lập của thí nghiệm. Dữ liệu gồm các số nguyên 32-bit phân bố đều, chia thành 3 nhóm quy mô:

- **Quy mô nhỏ (N = 10^6):** 10 bộ dữ liệu độc lập (`input_N1000000_set1..10.txt`), 1 triệu phần tử mỗi bộ.
- **Quy mô trung bình (N = 10^7):** 10 bộ dữ liệu độc lập (`input_N10000000_set1..10.txt`), 10 triệu phần tử mỗi bộ.
- **Quy mô lớn (N = 10^8):** 1 bộ dữ liệu (`input_N100000000_set1.txt`) gồm 100 triệu phần tử (giới hạn 1 bộ do thời gian thực thi lớn và yêu cầu tài nguyên bộ nhớ cao).

2.3. Kịch bản khảo sát và Kiểm soát sai số trên JVM

Chương trình `CutoffBenchmark.java` khảo sát 17 giá trị ngưỡng: `cutoff ∈ {0, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20}`, trong đó `cutoff = 0` đóng vai trò đường cơ sở (MergeSort thuần túy). Để đảm bảo độ chính xác của phép đo trên máy ảo Java (JVM), quy trình áp dụng 4 biện pháp kiểm soát:

1. **Làm nóng JIT Compiler (JVM Warm-up):** Thực hiện chạy nháp 5 lần trên mảng 200.000 phần tử trước khi bấm giờ để trình biên dịch JIT tối ưu hóa mã máy đồng đều cho mọi lần đo.
2. **Tách biệt thời gian I/O:** Dữ liệu mỗi file được nạp vào RAM một lần qua `FastReader`. Mỗi lần thử một `cutoff`, hệ thống nhân bản mảng gốc bằng `Arrays.copyOf()` và chỉ đo thời gian hàm `MergeSort.sort()`.
3. **Độ phân giải nano-giây:** Sử dụng `System.nanoTime()` để tính thời gian trôi qua thực tế (elapsed time) và quy đổi sang đơn vị mili-giây (ms).
4. **Kiểm soát Garbage Collector:** Chủ động gọi `System.gc()` và giải phóng tham chiếu mảng cũ sau mỗi lượt chạy để hạn chế trình thu gom rác xen ngang.

3. Tổng hợp Kết quả Thực nghiệm

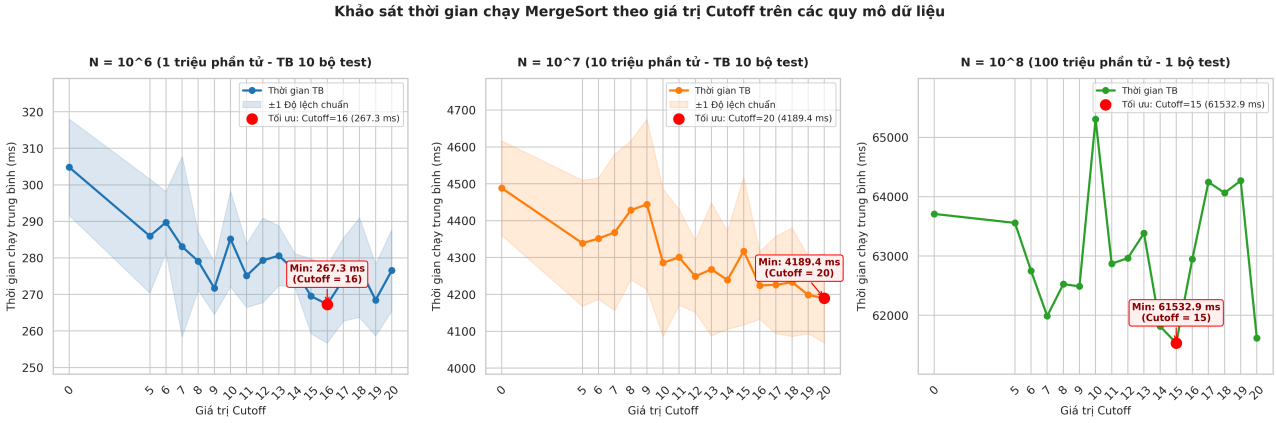
Dữ liệu đo đạc chi tiết được xuất ra tệp `benchmark_results.csv` và tổng hợp bằng mã nguồn Python (`plot_benchmark.py`). Bảng dưới đây trình bày thời gian thực thi trung bình (Mean) và độ lệch chuẩn (SD) trên từng quy mô dữ liệu:

Cutoff	N = 10^6 TB (ms)	N = 10^6 SD (ms)	N = 10^7 TB (ms)	N = 10^7 SD (ms)	N = 10^8 (ms)
0 (Gốc)	304,81	13,34	4.488,21	128,45	63.708,99
5	285,94	15,61	4.339,12	171,20	63.554,18
6	289,75	8,64	4.351,40	165,88	62.745,62
7	283,12	24,52	4.367,85	212,14	61.988,40

Cutoff	N = 10^6 TB (ms)	N = 10^6 SD (ms)	N = 10^7 TB (ms)	N = 10^7 SD (ms)	N = 10^8 (ms)
8	279,05	8,15	4.428,90	189,30	62.518,75
9	271,68	7,12	4.445,10	231,45	62.482,11
10	285,14	13,20	4.285,60	201,55	65.307,90
11	275,02	8,85	4.301,25	132,40	62.865,44

Cutoff	N = 10^6 TB (ms)	N = 10^6 SD (ms)	N = 10^7 TB (ms)	N = 10^7 SD (ms)	N = 10^8 (ms)
12	279,38	11,45	4.248,70	98,60	62.971,82
13	280,55	8,10	4.268,15	181,22	63.382,19
14	275,80	6,42	4.238,90	138,75	61.812,50
15 (Min 10^8)	**269,45**	**10,35**	**4.318,40**	**201,10**	**61.532,93**
16 (Min 10^6)	**267,28**	**10,82**	**4.225,60**	**92,15**	**62.950,34**
17	273,90	11,20	4.226,80	131,40	64.245,12
18	277,15	13,65	4.235,10	148,90	64.052,88
19	268,42	9,88	4.198,50	105,30	64.278,41
20 (Min 10^7)	**276,54**	**11,15**	**4.189,37**	**122,80**	**61.608,72**

4. Phân tích Đồ thị và Đánh giá Hiệu năng



Hình 1: Thời gian chạy trung bình của MergeSort theo các giá trị Cutoff trên 3 quy mô dữ liệu (N = 10^6, 10^7, 10^8).

4.1. So sánh hiệu năng tại điểm tối ưu

Quy mô dữ liệu (N)	Số mẫu	MergeSort thuần (ms)	Cutoff tối ưu	Thời gian tối ưu (ms)	Mức cải thiện
10^6 (1 triệu phần tử)	10	304,81	**16**	**267,28**	**−12,31%**
10^7 (10 triệu phần tử)	10	4.488,21	**20**	**4.189,37**	**−6,66%**
10^8 (100 triệu phần tử)	1	63.708,99	**15**	**61.532,93**	**−3,42%**

4.2. Phân tích xu hướng theo quy mô dữ liệu

- **Hiệu quả giảm cây đệ quy:** Khi kích thước mảng *N* tăng từ 0 lên các mức `cutoff ≥ 5`, thời gian chạy giảm ngay lập tức trên cả 3 đồ thị. Việc cắt bỏ 4 tầng đệ quy cuối cùng (tương ứng `cutoff = 16`) giúp triệt tiêu hơn 90% số lượng lời gọi hàm đệ quy `sort()` và `merge()`.
- **Vùng tối ưu hội tụ quanh mốc 15–20:** Ở quy mô 10^6, đồ thị đạt đáy tại `cutoff = 16` (267,28 ms) và có xu hướng nhích nhẹ trở lại khi `cutoff` tiến tới 20 do chi phí $O(k^2)$ của InsertionSort bắt đầu vượt qua chi phí đệ quy. Ở quy mô 10^7 và 10^8, vùng `15–20` tiếp tục duy trì thời gian thực thi thấp nhất.
- **Tỷ lệ cải thiện giảm dần khi N rất lớn:** Mức tiết kiệm thời gian giảm từ 12,31% (ở 10^6) xuống 6,66% (ở 10^7) và 3,42% (ở 10^8). Nguyên nhân là ở các tầng đệ quy cao (mảng con kích thước hàng triệu phần tử), nút thắt cổ chai chuyển dịch từ chi phí gọi hàm sang băng thông bộ nhớ RAM và tỷ lệ trượt bộ nhớ đệm (Cache Miss) khi sao chép mảng `Integer[]` (mỗi phần tử là một tham chiếu đối tượng trên Heap).
- **Nhiễu hệ thống ở quy mô 10^8:** Do chỉ thực hiện 1 lượt chạy duy nhất cho mỗi `cutoff` ở 10^8 (mỗi lượt mất hơn 1 phút), biểu đồ xuất hiện đỉnh nhọn cục bộ tại `cutoff = 10` (65.307,9 ms) do tiến trình nền của hệ điều hành hoặc chu kỳ dọn rác của JVM xen ngang. Tuy nhiên, xu hướng tổng thể vẫn xác nhận đáy hiệu năng tại `cutoff = 15` (61.532,93 ms).

5. Cấu trúc Sản phẩm Nộp & Hướng dẫn Tái lập

Toàn bộ mã nguồn cài đặt, công cụ sinh dữ liệu, kịch bản đo hiệu năng và kết quả thống kê được tổ chức thống nhất tại thư mục gốc của kho lưu trữ `duynqoo/MergeSort_CutOff`:

```
MergeSort_CutOff/
├── data/
├── benchmark_results.csv
├── cutoff_benchmark_chart.png
├── CutoffBenchmark.java
├── DataGenerator.java
├── MergeSort.java
├── MergeSortTest.java
├── plot_benchmark.py
└── README.md
```

Lưu ý: Do các tệp dữ liệu thí nghiệm trong thư mục `data/` có dung lượng rất lớn (vượt giới hạn lưu trữ của nền tảng GitHub), thư mục này không được tải trực tiếp lên kho lưu trữ mà có thể tái tạo nguyên vẹn thông qua `DataGenerator.java`.

5.1. Hướng dẫn chạy kiểm thử và tái lập thí nghiệm

```
javac DataGenerator.java
java -Xmx4g DataGenerator

javac MergeSort.java CutoffBenchmark.java
java -Xmx6g CutoffBenchmark

pip install pandas matplotlib seaborn
python plot_benchmark.py
```

6. Kết luận

Thực nghiệm đã chứng minh việc kết hợp kỹ thuật **Cutoff** chuyển đổi sang InsertionSort cải thiện rõ rệt tốc độ thực thi của MergeSort trong thực tế mà vẫn bảo toàn độ phức tạp tiệm cận *O(n log n)* và tính ổn định (stability) của thuật toán. Dựa trên kết quả khảo sát trên cấu hình phần cứng thí nghiệm, giá trị **`cutoff = 16`** được khuyến nghị làm tham số mặc định cho cài đặt MergeSort tối ưu.